

Probabilidade Discreta: Teoria, Teorema de Bayes, Valor Esperado e Variância na Tomada de Decisão

Gabriel M. P. Ribeiro*, Heitor E. M. Moreira**, Ingrid J. G. de Oliveira***,
Kleber M. da Cruz****, Rafael T. Dias*****

Área de Informática, Instituto Federal de Educação, Ciência e Tecnologia do
Estado de Minas Gerais (IFMG)
Rodovia MGT 262, KM 10, S/N - Bairro: Sobradinho
- CEP: 34.590-390

* Área de Informática, IFMG, Sabará, MG, (e-mail: gabrielmateusprribeiro@gmail.com)

** Área de Informática, IFMG, Sabará, MG, (e-mail: heitorleaoestudo@gmail.com)

*** Área de Informática, IFMG, Sabará, MG, (e-mail: ig.jeanine2@gmail.com)

**** Área de Informática, IFMG, Sabará, MG, (e-mail: kleber.martinsc@gmail.com)

***** Área de Informática, IFMG, Sabará, MG, (e-mail: rafaelteixeiradias946@gmail.com)

Abstract. *The objective of this work is to analyze the practical interaction between Probability Theory, Bayes' Theorem, Expected Value, and Variance in decision-making under uncertainty. To achieve this, a theoretical bibliographic review was conducted alongside a practical computational simulation to model the behavior and risk of discrete events. The results demonstrate that these four statistical concepts do not operate in isolation, proving that they form a sequential and strictly dependent flow for the safe and assertive analysis of probabilistic scenarios.*

Resumo. *O objetivo deste trabalho é analisar a interação prática entre a Teoria da Probabilidade, o Teorema de Bayes, o Valor Esperado e a Variância na tomada de decisões sob incerteza. Para isso, foi realizada uma revisão teórica bibliográfica aliada a uma simulação computacional prática para modelar o comportamento e o risco de eventos discretos. Os resultados demonstram que esses quatro conceitos estatísticos não operam isoladamente, comprovando que formam um fluxo sequencial e dependente para a análise segura e assertiva de cenários probabilísticos.*

1. Introdução

A Teoria da Probabilidade é um dos principais ramos da Matemática Discreta voltados ao estudo de eventos aleatórios em espaços amostrais finitos ou contáveis. Seu objetivo é quantificar a chance de ocorrência de determinados eventos por meio de modelos matemáticos, permitindo que situações incertas sejam analisadas de forma lógica e objetiva. Essa teoria possui aplicações em diversas áreas, como computação, inteligência artificial, ciência de dados, economia, engenharia e segurança da informação, sendo um importante instrumento para a tomada de decisões fundamentadas.

Na Matemática Discreta, a probabilidade está diretamente relacionada à análise combinatória e aos conjuntos, pois muitos problemas envolvem a contagem de possibilidades para determinar a probabilidade de um evento ocorrer. A abordagem clássica define

a probabilidade como a razão entre o número de casos favoráveis (m) e o número total de casos possíveis (n), desde que todos possuam a mesma chance de ocorrência:

$$P(A) = \frac{m}{n}$$

Essa definição constitui a base para o desenvolvimento de técnicas probabilísticas mais avançadas. Historicamente, o estudo formal da probabilidade teve início no século XVII com a análise de jogos de azar e trocas de correspondências entre matemáticos renomados como Blaise Pascal e Pierre de Fermat. Desde então, a área evoluiu significativamente, deixando de ser apenas um método empírico de estimativa de apostas para se consolidar como um ramo rigoroso da matemática pura e aplicada, formalizado axiomaticamente por Andrey Kolmogorov na década de 1930. A probabilidade discreta, em especial, desempenha um papel crucial no desenvolvimento de algoritmos modernos, permitindo lidar com a incerteza inerente a dados complexos e fundamentando técnicas de busca, modelagem estatística, criptografia e inteligência artificial baseada em inferência.

Este trabalho tem como objetivo apresentar os fundamentos da Teoria da Probabilidade e seus principais desdobramentos conceituais no contexto de modelos discretos, especificamente o Valor Esperado, o Teorema de Bayes e as Medidas de Dispersão (Variância e Desvio Padrão). A metodologia adotada para este estudo envolve a discussão detalhada das definições teóricas clássicas dessas métricas acoplada ao desenvolvimento de implementações práticas e simulações estocásticas nas linguagens Java e Python. Dessa maneira, busca-se demonstrar empiricamente a aplicabilidade das formulações matemáticas na resolução de problemas práticos de engenharia de software e tomada de decisões sob condições de incerteza, validando a teoria com observações computacionais.

Os resultados obtidos a partir das simulações computacionais evidenciam a convergência das frequências relativas empíricas aos valores analíticos deduzidos teoricamente, em conformidade com a Lei dos Grandes Números. O experimento clássico de lançamentos repetidos de moedas comprova a estabilidade da probabilidade de eventos independentes a longo prazo, enquanto a simulação do Teorema de Bayes ilustra a robustez da inferência bayesiana na atualização de hipóteses clínicas na presença de novos dados de exames. Adicionalmente, as análises sobre medidas de variabilidade demonstram a importância prática da estabilidade numérica, evidenciando como abordagens incrementais mitigam erros de cancelamento catastrófico em fluxos de dados contínuos.

Para apresentar esses tópicos de forma clara e fluida, este relatório acadêmico está estruturado em seções bem definidas. A Seção 2 descreve os conceitos elementares de probabilidade discreta e introduz a simulação Java do jogo clássico de Cara ou Coroa. A Seção 3 discute o Valor Esperado e apresenta a plotagem em Python para visualização de sua convergência assintótica. A Seção 4 detalha as bases teóricas do Teorema de Bayes e expõe um algoritmo de simulação probabilística aplicado a diagnósticos. A Seção 5 analisa as Medidas de Dispersão, como Variância e Desvio Padrão, contrapondo o método direto tradicional e a recursividade do Algoritmo de Welford. Por fim, a Seção 8 discute os resultados obtidos e a Seção 10 consolida as conclusões finais deste estudo.

2. Probabilidade

Para a realização do primeiro experimento prático, utilizou-se o modelo clássico do lançamento de moedas, especificamente o jogo de Cara ou Coroa. Neste contexto, define-se um espaço amostral discreto e equiprovável $\Omega = \{\text{Cara}, \text{Coroa}\}$, no qual cada evento simples possui a mesma probabilidade de ocorrência. Dessa forma, a probabilidade teórica de obter a face "Cara" em um único lançamento de uma moeda justa é dada por:

$$P(\text{Cara}) = \frac{1}{2} = 0,5 \quad (1)$$

Estendendo o experimento para o lançamento simultâneo de três moedas honestas e independentes, o espaço amostral resultante passa a conter $2^3 = 8$ eventos possíveis de igual probabilidade. A probabilidade de obter a face "Cara" em todos os três lançamentos é calculada pelo produto das probabilidades individuais, visto que os lançamentos constituem eventos mutuamente independentes:

$$P(\text{Cara}, \text{Cara}, \text{Cara}) = \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} = 0,125 \quad (2)$$

A fim de simular esse comportamento estocástico de forma computacional, a implementação desenvolvida realiza o experimento de lançamento das três moedas por um total de n iterações, calculando a probabilidade experimental de sucesso (o total de vezes que o evento triplo ocorreu dividido pelo total de testes). Após a repetição de múltiplos experimentos independentes, são calculadas a média aritmética e a variância dos resultados experimentais obtidos. A seguir, é apresentada a implementação em linguagem Java criada para conduzir essa simulação:

```
1
2 import java.util.Random;
3
4 public class Probabilidade {
5     public static void main(String[] args) {
6         double media = 0, variancia = 0, soma_desvio = 0;
7         int quant_experimentos = 10;
8         double eventos_total = 100;
9         Random gerador = new Random();
10
11         // cria um vetor para guardar a probabilidade de cada
12         // experimento
13         double[] probabilidade = new double[quant_experimentos];
14
15         for (int i = 0; i < quant_experimentos; i++) {
16             // realiza o experimento i
17             int eventos_sucesso = 0;
18             for (int j = 0; j < eventos_total; j++) {
19                 // gerar 3 valores aleatórios de 0 ou 1
20                 // 0 = cara, 1 = coroa
21                 int r1 = gerador.nextInt(2);
22                 int r2 = gerador.nextInt(2);
23                 int r3 = gerador.nextInt(2);
24
25                 // verifica se as 3 moedas deram cara
```

```

25         if (r1 == 0 && r2 == 0 && r3 == 0) {
26             eventos_sucesso++;
27         }
28     }
29     // calcula a probabilidade experimental e guarda esse valor
30     probabilidade[i] = eventos_sucesso / eventos_total;
31     // soma para fazer a média
32     media += probabilidade[i];
33 }
34
35 // calcula a média
36 media = media / quant_experimentos;
37
38 // calcula a variância
39 for (int i = 0; i < quant_experimentos; i++) {
40     double desvio = probabilidade[i] - media;
41     desvio = desvio * desvio;
42     soma_desvio += desvio;
43 }
44
45 // calcula a variância usando os desvios
46 variancia = soma_desvio / quant_experimentos;
47
48 // indica as métricas
49 System.out.println("Métricas:");
50 System.out.println("Para " + quant_experimentos + "
51 experimentos, com " + eventos_total + " eventos cada");
52 System.out.println("Média: " + media);
53 System.out.println("Variância: " + variancia);
54 }

```

3. Valor Esperado

Outro conceito fundamental da probabilidade discreta é o valor esperado, também conhecido como esperança matemática. Esse conceito representa a média dos resultados de uma variável aleatória caso um experimento seja repetido um grande número de vezes nas mesmas condições. Em outras palavras, o valor esperado indica o resultado médio de longo prazo, mesmo que esse valor nem sempre corresponda a um resultado que possa ser observado individualmente.

Para uma variável aleatória discreta, o valor esperado é calculado pela soma dos produtos entre cada valor possível da variável e sua respectiva probabilidade de ocorrência. Utiliza-se a expressão:

$$E(X) = \sum X_i P(X_i) \quad (3)$$

em que X_i representa cada valor possível assumido pela variável aleatória e $P(X_i)$ corresponde à probabilidade associada a esse valor. Esse cálculo leva em consideração todos os resultados possíveis e seus respectivos pesos probabilísticos, produzindo uma medida representativa do comportamento médio da variável.

O conceito de valor esperado possui ampla aplicação em economia, finanças, engenharia, ciência de dados e teoria dos jogos. Em jogos de azar, por exemplo, ele permite avaliar se uma aposta tende a gerar lucro ou prejuízo ao longo do tempo. Da mesma forma, em problemas de tomada de decisão, o valor esperado auxilia na comparação entre diferentes alternativas sob condições de incerteza, tornando-se uma importante ferramenta para análise quantitativa e planejamento baseado em probabilidades.

```
1
2 import random
3
4 import matplotlib.pyplot as plt
5 import numpy as np
6
7 elements = 10**6
8
9 weights = [50, 20, 10, 20, 30, 40]
10 values = [-100, -500, 500, 400, 300, 200]
11 weight_total = sum(weights)
12
13 expected_value = 0
14 for i in range(len(values)):
15     expected_value += ((weights[i] / weight_total) * values[i])
16
17 x = np.linspace(1, elements, elements)
18 value = random.choices(population=values, weights=weights, k=elements)
19 y = np.cumsum(value)
20
21 plt.rc('lines', linewidth=2.5)
22 fig, ax = plt.subplots()
23 ax.set_title('Valor Esperado')
24 ax.grid(True)
25 ax.set_xscale('log')
26 ax.set_yscale('symlog')
27
28 line1, = ax.plot(x, expected_value * x, dashes=[6, 2], label='valor
    esperado')
29 line2, = ax.plot(x, y, label='valor gerado')
30
31 ax.legend(handlelength=4)
32 plt.show()
33
34 plt.show()
```

4. Teorema de Bayes

O Teorema de Bayes, resultado dos estudos matemáticos de Thomas Bayes e Pierre-Simon Laplace, é um teorema que permite atualizar a probabilidade de um evento a partir de novas informações disponíveis. Além disso, ele possibilita calcular a probabilidade de uma hipótese ser verdadeira sabendo que determinado evento ocorreu, mesmo quando se conhece apenas a probabilidade de esse evento ocorrer caso a hipótese seja verdadeira. Por essa razão, o teorema é amplamente utilizado em situações que envolvem tomada de decisão sob incerteza e interpretação de evidências.

Matematicamente, o Teorema de Bayes pode ser expresso pela fórmula:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)} \quad (4)$$

Cada um dos termos presentes na Equação 4 possui uma denominação formal e um significado específico dentro da inferência estatística:

- $P(A|B)$ é a **probabilidade a posteriori** (ou posterior): representa a probabilidade atualizada de a hipótese A ser verdadeira após a observação da evidência B .
- $P(B|A)$ é a **verossimilhança** (*likelihood*): corresponde à probabilidade de observar a evidência B dado que a hipótese A é verdadeira.
- $P(A)$ é a **probabilidade a priori** (ou prior): indica a probabilidade inicial atribuída à hipótese A antes de qualquer nova evidência ser coletada.
- $P(B)$ é a **probabilidade marginal** (ou evidência): expressa a probabilidade total de a evidência B ocorrer sob todos os cenários possíveis do espaço amostral.

Frequentemente, a probabilidade marginal $P(B)$ não é conhecida de antemão. Para obtê-la, aplica-se a **Lei da Probabilidade Total**. Se considerarmos uma hipótese A e o seu complemento $\bar{A}$ (o evento em que a hipótese é falsa), que juntos formam uma partição do espaço amostral, a evidência $P(B)$ pode ser calculada como:

$$P(B) = P(B|A)P(A) + P(B|\bar{A})P(\bar{A}) \quad (5)$$

Substituindo a Equação 5 na formulação original (Equação 4), obtém-se a forma expandida do Teorema de Bayes:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B|A)P(A) + P(B|\bar{A})P(\bar{A})} \quad (6)$$

Esse processo de atualização é fundamental para análises científicas e tomadas de decisão. O matemático Harold Jeffreys descreveu o Teorema de Bayes como o correspondente na probabilidade ao que o Teorema de Pitágoras representa para a geometria clássica.

Para ilustrar o teorema de maneira intuitiva e evidenciar o seu caráter às vezes contra-intuitivo, considere um cenário de diagnóstico clínico de uma doença rara. Suponha que essa patologia acometa apenas 1% de determinada população, o que nos dá uma probabilidade a priori de ter a doença igual a $P(\text{Doença}) = 0,01$. Um laboratório desenvolveu um teste para detectar a enfermidade, cujo exame apresenta uma sensibilidade (taxa de verdadeiro positivo) de 95%, ou seja, $P(\text{Positivo}|\text{Doença}) = 0,95$. No entanto, o teste possui uma taxa de falso positivo de 5%, significando que $P(\text{Positivo}|\text{Saudável}) = 0,05$.

Se um paciente escolhido ao acaso nessa população realiza o teste e o resultado é positivo, qual é a probabilidade real de que ele de fato possua a doença? Aplicando o teorema expandido (Equação 6), calcula-se primeiramente a probabilidade marginal de obter um resultado positivo:

$$P(\text{Positivo}) = P(\text{Positivo}|\text{Doença})P(\text{Doença}) + P(\text{Positivo}|\text{Saudável})P(\text{Saudável})$$

$$P(\text{Positivo}) = (0,95 \cdot 0,01) + (0,05 \cdot 0,99) = 0,0095 + 0,0495 = 0,0590$$

Agora, calcula-se a probabilidade posterior do paciente estar doente dado o teste positivo:

$$P(\text{Doença}|\text{Positivo}) = \frac{P(\text{Positivo}|\text{Doença}) \cdot P(\text{Doença})}{P(\text{Positivo})} = \frac{0,0095}{0,0590} \approx 0,1610 \quad (7)$$

O resultado revela que o indivíduo que obteve o teste positivo tem apenas cerca de 16,10% de chance de estar realmente doente. Esse fenômeno decorre do fato de a doença ser extremamente rara na população (1%), fazendo com que a quantidade absoluta de falsos positivos gerados pelo teste seja muito superior à quantidade de verdadeiros positivos.

Para validar numericamente esse comportamento, o script em Python a seguir realiza tanto o cálculo analítico exato quanto uma simulação empírica baseada na geração randômica de uma população virtual de indivíduos, comparando os resultados teóricos com os obtidos por simulação:

```
1 import random
2
3 # Exemplo: Diagnostico de uma Doença Rara
4 # P(D) = Probabilidade a priori de ter a doença
5 # P(T|D) = Sensibilidade (teste positivo dado que tem a doença)
6 # P(T|S) = Falso positivo (teste positivo dado que está saudável)
7
8 prevalencia = 0.01      # P(Doença) = 1%
9 sensibilidade = 0.95    # P(Positivo|Doença) = 95%
10 falso_positivo = 0.05  # P(Positivo|Saudavel) = 5%
11
12 # --- Calculo Analitico pelo Teorema de Bayes ---
13 prob_saudavel = 1.0 - prevalencia
14 prob_evidencia = (sensibilidade * prevalencia) + (falso_positivo *
15     prob_saudavel)
16 prob_posterior_teorica = (sensibilidade * prevalencia) / prob_evidencia
17
18 print("--- Calculo Analitico ---")
19 print(f"P(Doença|Positivo) Teorica: {prob_posterior_teorica:.4f} ({
20     prob_posterior_teorica:.2%})")
21
22 # --- Simulacao Empirica (Geracao Randomica) ---
23 populacao_tamanho = 100000
24 verdadeiros_positivos = 0
25 falsos_positivos = 0
26
27 for _ in range(populacao_tamanho):
28     # Determina se o individuo tem a doença com base na prevalencia
29     tem_doenca = random.random() < prevalencia
30
31     if tem_doenca:
32         # Se tem a doença, verifica se o teste da positivo (
33         # sensibilidade)
34         if random.random() < sensibilidade:
35             verdadeiros_positivos += 1
36     else:
37         # Se nao tem a doença, verifica se ocorre falso positivo
38         if random.random() < falso_positivo:
39             falsos_positivos += 1
```

```

37
38 total_positivos = verdadeiros_positivos + falsos_positivos
39 prob_posterior_empirica = verdadeiros_positivos / total_positivos if
    total_positivos > 0 else 0
40
41 print("\n--- Resultados da Simulacao Empirica ---")
42 print(f"Populacao Total Simulada: {populacao_tamanho}")
43 print(f"Total de Testes Positivos Obtidos: {total_positivos}")
44 print(f" - Verdadeiros Positivos (Doentes reais): {
    verdadeiros_positivos}")
45 print(f" - Falsos Positivos (Saudaveis com teste pos.): {
    falsos_positivos}")
46 print(f"P(Doenca|Positivo) Empirica: {prob_posterior_empirica:.4f} ({
    prob_posterior_empirica:.2%})")
47 print(f"Diferenca Absoluta: {abs(prob_posterior_teorica -
    prob_posterior_empirica):.6f}")

```

5. Medidas de Dispersão

As medidas de dispersão, ou variabilidade estatística, consistem basicamente em mostrar o quão uma distribuição de valores estão dispersas em relação à média ou ao valor esperado, tendo a possibilidade de haver amostras com valores geralmente mais próximos ou geralmente mais afastados. O fato dos valores serem todos, ou majoritariamente, diferentes entre si muda a forma como se deve lidar com eles ou com a cadeia de valores como um todo. Isso se dá ao fato de que, dentre os diferentes tipos de variabilidade, podem possuir nas amostras comportamentos idênticos, comportamentos meramente semelhantes, ou comportamentos completamente distintos um do outro, não podendo tratar tais casos da mesma forma. Ou seja, a média que é considerada como um número que representa uma série de valores não pode, por si mesma, destacar o grau de homogeneidade ou de heterogeneidade que há entre eles.

Um exemplo prático disso são os dados coletados através dos resultados de uma prova em diferentes turmas. Supomos que a média geral das notas da prova, em ambas as turmas, tenha sido nota 5, porém, na primeira sala houveram tanto alunos que tiraram 10, como alunos que tiraram 0. Enquanto isso, na segunda turma, as notas variam apenas entre 6 e 4. Se o professor olhar unicamente para a média e usá-la como dado para futuras alterações em sua metodologia, tal alteração não será efetiva. Isso se dá pelo fato de que as turmas sofreram de diferentes variações de valores, o que implica em diferentes extremos de desempenho. Sendo assim, uma forma de se analisar a situação seria, por exemplo, levar em conta a primeira turma, com uma variação entre 0 e 10, e dar a devida ajuda e mentoria aos alunos que tiraram 0, enquanto incentiva os alunos que tiraram 10. Enquanto isso, na segunda turma, o fato da variação estar entre 4 e 6 poderia implicar em, por exemplo, erros dentro da própria prova, como a existência de questões exageradamente difíceis, longas ou com conteúdo não passado.

Em um âmbito matemático, é possível analisar, por exemplo, os conjuntos de valores a seguir:

- A = {60, 60, 60, 60, 60}
- B = {58, 59, 60, 61, 62}
- C = {5, 10, 40, 100, 145}

Ao calcularmos a mesma média aritmética desses conjuntos:

$$\bar{a} = \frac{\sum a_i}{n} = \frac{300}{5} = 60 \quad (8)$$

$$\bar{b} = \frac{\sum b_i}{n} = \frac{300}{5} = 60 \quad (9)$$

$$\bar{c} = \frac{\sum c_i}{n} = \frac{300}{5} = 60 \quad (10)$$

Verificamos que os três conjuntos apresentam a mesma média aritmética: 60. Chegamos à conclusão que o conjunto A é mais homogêneo que os conjuntos B e C, visto que todos os valores são iguais à média. O conjunto B, por sua vez, é mais homogêneo que o conjunto C, pois há menor diversificação entre cada um de seus valores e a média representativa. Logo, o conjunto A apresenta dispersão nula, e o conjunto B apresenta uma dispersão menor que C.

Dentre as medidas de dispersão, é muito comum a utilização da Variância e do Desvio padrão.

6. Variância

A variância, também conhecida como Desvio Quadrático Médio, mede a dispersão dos dados em torno de sua média, levando em consideração a totalidade dos valores da variável em estudo, o que a torna um índice de variabilidade bastante estável. A variância é representada por s^2 e definida como sendo a média dos quadrados dos desvios em relação à média aritmética.

Sua fórmula é representada por:

$$s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1} \quad (11)$$

Podendo, também, ser destrinchada em:

$$s^2 = \frac{(x_1 - \bar{x})^2 + (x_2 - \bar{x})^2 + \dots + (x_n - \bar{x})^2}{n - 1} \quad (12)$$

Essa fórmula consiste, basicamente, em somar a distância de cada amostra em relação à média geral, explicando, assim, o termo: $(x_1 - \bar{x}) + \dots + (x_i - \bar{x})$. Logo depois, é dividido pela quantidade de termos, fazendo-se assim a média dos afastamentos: $\frac{(x_1 - \bar{x}) + \dots + (x_i - \bar{x})}{n-1}$. Porém, feito desse jeito, valores individuais que forem menores que a média geral, caso subtraídos a essa média, eles ficarão negativos, o que criaria um cenário onde os valores, no fim se anulariam, a fórmula total chegaria próxima a 0. Para isso, é necessário elevar o resultado das subtrações ao quadrado, para assim todos os valores fiquem positivos: $\frac{(x_1 - \bar{x})^2 + (x_2 - \bar{x})^2 + \dots + (x_n - \bar{x})^2}{n-1}$. E, por fim, para encurtar a fórmula ao máximo, traduzimos a série de somas com uma somatória, que começa do 1 e vai até o n: $s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n-1}$.

Com isso, o valor da variância se dá por s^2 em razão à série de elevações ao quadrado que se tem dentro da somatória, porém, se ao final for optado por colocar o resultado dentro de uma raiz quadrada, obteremos o Desvio Padrão.

6.1. Outras fórmulas

Em segundo caso, Quando a média é um valor decimal não exato, a fórmula da variância apresentada anteriormente pode se tornar não muito prática, uma vez que entrará no cálculo “n” vezes aumentando os erros de arredondamento que ocorrem. Então, é recomendável utilizar uma expressão alternativa que é obtida através de algumas manipulações algébricas na fórmula anterior:

$$s^2 = \frac{1}{n-1} \left[\left(\sum_{i=1}^n (x_i^2) \right) - n(\bar{x})^2 \right] \quad (13)$$

Além disso, existe, também, a fórmula de cálculo, onde simplifica o processo, pois evita calcular o desvio de cada valor em relação à média:

$$s^2 = \frac{\sum x_i^2}{n} - \bar{x}^2 \quad (14)$$

6.2. Demonstrando fórmula de cálculo

É possível provar a veracidade da segunda fórmula seguindo os passos do método de indução ou por comparação de valores. Sendo assim, digamos que em uma comunidade há 5 pessoas, sendo que cada uma delas possui uma altura diferente: 150 cm, 160 cm, 170 cm, 180 cm e 190 cm.

6.2.1. Fórmula original

1. Calcular média:

$$\bar{x} = \frac{150 + 160 + 170 + 180 + 190}{5} = 170cm$$

2. Calcular a diferença de cada valor para a média e elevar ao quadrado:

$$s^2 = \frac{(150 - 170)^2 + (160 - 170)^2 + (170 - 170)^2 + (180 - 170)^2 + (190 - 170)^2}{5} =$$

$$s^2 = \frac{(-20)^2 + (-10)^2 + 0^2 + 10^2 + 20^2}{5} = \frac{400 + 100 + 0 + 100 + 400}{5}$$

3. Somar todos os itens e dividir

$$s^2 = \frac{400 + 100 + 0 + 100 + 400}{5} = \frac{1000}{5} = 200cm^2$$

6.2.2. Fórmula alternativa

1. Elevar os termos

$$s^2 = \frac{150^2 + 160^2 + 170^2 + 180^2 + 190^2}{5} = \frac{22500 + 25600 + 28900 + 32400 + 36100}{5}$$

2. Somar os termos e subtrair

$$s^2 = \frac{22500 + 25600 + 28900 + 32400 + 36100}{5} = \frac{145500}{5} = 29100$$

3. Finalizar com o resto da fórmula

$$s^2 = 29100 - 170^2 = 29100 - 28900 = 200cm^2$$

6.3. Variância amostral e populacional

O conceito de variância também pode ser usado para descrever um conjunto de observações. Ela é separada em dois tipos: variância da amostra e variância da população.

A variância populacional indica como os pontos de dados de uma população como um todo estão dispersos. Já a variância amostral é usada para calcular a variabilidade em uma determinada amostra. Uma amostra é um conjunto de observações extraídas de uma população e que a representam completamente.

$$\text{Amostral: } s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1} \quad (15)$$

$$\text{Populacional: } \sigma^2 = \frac{\sum_{i=1}^n f(x_i - \bar{x})^2}{N} \quad (16)$$

Ambas se diferenciam pela letra grega Sigma (σ) e pela letra s. Além, também, da diferenciação através da divisão, onde em um é com n-1, representando o número de elementos da amostra, e outro é com N apenas, representando o número total de elementos.

6.4. Variância da amostra: agrupada e não agrupada

Como os dados podem ser de dois tipos, agrupados e não agrupados, existem duas fórmulas disponíveis para calcular a variância da amostra.

Como exemplo, suponha que um conjunto de dados seja dado como 3, 21, 98, 17 e 9. A média (29,6) do conjunto de dados é determinada. A média é subtraída de cada ponto de dados e a soma dos quadrados dos valores resultantes é calculada. Isso resulta em 6043,2. Para obter a variância da amostra, esse número é dividido por um a menos que o número total de observações. Assim, a variância da amostra é 1510,8.

Os dados, quando estão em formato bruto e desorganizado, são conhecidos como dados não agrupados. Já os dados agrupados são classificados em categorias ou tabelas. As fórmulas de variância amostral para ambos os tipos de dados são dadas de maneiras diferentes:

$$\text{Desagrupados: } s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1} \quad (17)$$

$$\text{Agrupados: } s^2 = \frac{\sum_{i=1}^n f_i (x_i - \bar{x})^2}{n - 1} \quad (18)$$

Como é possível analisar, a diferença se dá pelo f_i na fórmula dos dados desagrupados, sendo uma variável que representa a frequência absoluta da i -ésima classe (quantos dados caem nela).

6.5. Abordagens Computacionais para Cálculo de Variância

A implementação do cálculo da variância em sistemas computacionais pode ser feita por meio de diferentes estratégias. A abordagem tradicional (de duas passagens) calcula primeiramente a média aritmética de todo o conjunto de dados para, em uma segunda iteração, realizar a soma dos desvios quadráticos de cada elemento.

Por outro lado, o Algoritmo de Welford é um método matemático e computacional que calcula a média e a variância de forma incremental e numericamente estável, utilizando formulações recursivas. Ele permite que estatísticas sejam extraídas em tempo real conforme os dados são fornecidos, sem a necessidade de armazenar todos os valores na memória ou realizar múltiplas passagens pelos dados. A cada nova entrada de dado x_n , a média $\bar{x}_n$ é atualizada de acordo com a seguinte recorrência:

$$\bar{x}_n = \bar{x}_{n-1} + \frac{x_n - \bar{x}_{n-1}}{n} \quad (19)$$

Abaixo é apresentada a implementação em Python, contendo tanto a função tradicional de duas passagens quanto a abordagem estável de Welford:

```

1
2 def varianciaPadrao(amostras):
3     media = 0 # Média dos valores
4     valores = 0
5     N = len(amostras) # Quantidade de amostras
6
7     # loop que preenche a media
8     for x in amostras:
9         media = media + x
10
11     # Cálculo de média
12     media = media / N
13
14     # Iteração que efetua a soma dos valores
15     for x in amostras:
16         valores = valores + (x - media) ** 2
17
18     return valores / N
19
20
21 # Função pra calcular media em recursão
22 # O índice, nesse caso, representa a quantidade de itens

```

```

23 def mediaWelford(amostras, indice):
24     # Se o valor for o primeiro, retorna ele, pois ele já é a média
    dele mesmo, se não, continua a recursão
25     if indice == 0:
26         return amostras[indice]
27     else:
28         mediaAnterior = mediaWelford(amostras, indice - 1)
29         return mediaAnterior + (amostras[indice] - mediaAnterior) / (
    indice + 1)
30
31
32 def valoresWelford(amostras, indice, media):
33     # Se o valor for o primeiro, retorna o quadrado dele, se não,
    continua com o cálculo da recursão
34     if indice == 0:
35         return (amostras[indice] - media) * (amostras[indice] - media)
36     else:
37         return valoresWelford(amostras, indice - 1, media) + (amostras[
    indice] - media) * (amostras[indice] - media)
38
39
40 def varianciaWelford(amostras):
41     # Armazena a média
42     media = mediaWelford(amostras, len(amostras) - 1)
43
44     # Armazena os valores ao quadrado somados
45     somaDesviosQuadrados = valoresWelford(amostras, len(amostras) - 1,
    media)
46
47     # Resultado da variância
48     return somaDesviosQuadrados / len(amostras)
49
50
51 # Calculando...
52 # lista = [15, 29, 34, 40]
53 # print(varianciaPadrao(lista)) # Saída: 85.25
54 # print(varianciaWelford(lista)) # Saída: 85.25

```

A mesma lógica de implementação estruturada em linguagem Java, contendo ambos os métodos para comparação de resultados, é apresentada a seguir:

```

1
2 public class VarianceCalculator {
3
4     // Calcula a variância pelo método tradicional.
5     public static float calculateStandardVariance(int[] samples) {
6         int sampleCount = samples.length;
7         float mean = 0;
8
9         // Calcula a média da amostra.
10        for (int sample : samples) {
11            mean += sample;
12        }
13
14        mean /= sampleCount;
15

```

```

16         float squaredDifferenceSum = 0;
17
18         // Soma os desvios ao quadrado em relação à média.
19         for (int sample : samples) {
20             float difference = sample - mean;
21             squaredDifferenceSum += difference * difference;
22         }
23
24         return squaredDifferenceSum / sampleCount;
25     }
26
27     // Calcula a média da amostra de forma recursiva.
28     private static float calculateWelfordMean(int[] samples, int index)
29     {
30         // Caso base: o primeiro elemento é a média dele mesmo.
31         if (index == 0) {
32             return samples[0];
33         }
34
35         float previousMean = calculateWelfordMean(samples, index - 1);
36
37         // Atualiza a média considerando o novo elemento.
38         return previousMean + (samples[index] - previousMean) / (index
39 + 1);
40     }
41
42     // Soma recursivamente os desvios ao quadrado.
43     private static float calculateSquaredDifferenceSum(int[] samples,
44 int index, float mean) {
45         float difference = samples[index] - mean;
46
47         // Caso base da recursão.
48         if (index == 0) {
49             return difference * difference;
50         }
51
52         return calculateSquaredDifferenceSum(samples, index - 1, mean)
53 + difference * difference;
54     }
55
56     // Calcula a variância utilizando os métodos recursivos.
57     public static float calculateWelfordVariance(int[] samples) {
58         int sampleCount = samples.length;
59
60         float mean = calculateWelfordMean(samples, sampleCount - 1);
61
62         float squaredDifferenceSum =
63             calculateSquaredDifferenceSum(samples, sampleCount - 1,
64 mean);
65
66         return squaredDifferenceSum / sampleCount;
67     }
68
69     public static void main(String[] args) {
70         int[] samples = {15, 29, 34, 40};
71     }

```

```

68     System.out.println(calculateStandardVariance(samples)); // Saída
    da 85.25
69     System.out.println(calculateWelfordVariance(samples)); // Saída
    85.25
70 }
71 }

```

Esse algoritmo incremental é muito menos propenso à perda de precisão decorrente de cancelamento catastrófico em ponto flutuante, sendo ideal para processamento de fluxos contínuos de dados (*streaming*).

7. Desvio Padrão

Em comparação à variância, é a medida de dispersão geralmente mais empregada, pois leva em consideração a totalidade dos valores da variável em estudo. O desvio padrão é uma medida de dispersão onde também se faz uso da média geral, medindo a variação dos valores ao redor dela. O valor mínimo do desvio padrão é 0, indicando que não há variabilidade, ou seja, que todos os valores são iguais à média. O símbolo para o desvio padrão em um conjunto de dados observados é s , e a fórmula para obter o desvio padrão é dada pela fórmula da variância, porém, implementada dentro de uma raiz quadrada, tendo em visto que a variância eleva seus valores à 2:

$$s = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n - 1}} \quad (20)$$

7.1. Propriedades de Desvio Padrão

- Ao adicionarmos ou subtraírmos uma constante a todos os valores de uma variável, o desvio padrão não se altera.

- Ao multiplicarmos ou dividirmos todos os valores de uma variável por uma constante (diferente de zero), o desvio padrão fica multiplicado (ou dividido) por essa constante.

- Tal como a variância altera sua fórmula quando os dados estão agrupados, o mesmo é feito dentro do desvio padrão: $s = \sqrt{\frac{\sum f_i x_i^2}{n} - (\frac{\sum f_i x_i}{n})^2}$

7.2. Coeficiente de variação

O desvio padrão é uma medida limitada se utilizada isoladamente. Por exemplo, um desvio padrão de 2 unidades pode ser considerado pequeno para uma série de valores cujo valor médio é 200; no entanto, se a média for igual a 20, o mesmo não pode ser dito. Quando desejamos comparar duas ou mais unidades relativamente à sua dispersão ou variabilidade, o desvio padrão não é a medida mais indicada, visto que ele se encontra na mesma unidade dos dados.

Assim, contornamos este problema caracterizando a dispersão em termos relativos ao seu valor médio. Essa medida é denominada de coeficiente de variação de Pearson. O coeficiente de variação de Pearson é a razão entre o desvio padrão e a média referentes aos dados de uma mesma série.

O coeficiente de variação de pearson é a razão entre o desvio padrão e a média referentes aos dados de uma mesma série.

$$CV = \frac{s}{x}$$

8. Discussão e Análise de Resultados

Para o valor esperado, foi elaborada uma comparação gráfica para mostrar como os valores se comportam a partir da Lei dos Grandes Números. Primeiramente, o código foi elaborado em Python para melhor análise de dados, e foram necessários importar bibliotecas, tais como `random` (usada para gerar valores aleatórios), `matplotlib.pyplot` (usada para criar gráficos e visualizar dados) e `numpy` (usada para cálculos matemáticos e científicos):

```
1 import random
2 import matplotlib.pyplot as plt
3 import numpy as np
```

Adiante, as variáveis principais para o cálculo foram criadas, juntamente da quantidade de iterações que seriam feitas:

```
1 elements = 10**6 # Um milhão de simulações
2 weights = [50, 20, 10, 20, 30, 40] # Peso de cada valor
3 values = [-100, -500, 500, 400, 300, 200] # Valores
4 weightTotal = sum(weights) # Armazena os pesos somados
```

Assim podemos seguir para o script de cálculo matemático, utilizando-se da fórmula geral, além também de gerar uma série de valores aleatórios como entrada para a comparação base:

```
1 # Cálculo do valor esperado
2 expectedValue = 0
3 for i in range(len(values)):
4     # Calcula probabilidade e multiplica pelo valor, isso em cada elemento
5     expectedValue += ((weights[i] / weightTotal) * values[i])
6
7 # Gerando valores aleatórios pra comparar com o valor esperado
8 x = np.linspace(1, elements, elements)
9 valueArrayRandom = random.choices(population=values, weights=weights, k=elements) # Sorteia 1 milhão de valores
10 y = np.cumsum(valueArrayRandom)
```

Por último, o código para a criação dos gráficos foram criados, junto com sua chamada, utilizando-se dos valores randomizados:

```
1 # Implementação de gráfico
2 plt.rc('lines', linewidth=2.5)
3 fig, ax = plt.subplots()
4 ax.set_title('Valor Esperado')
5 ax.grid(True)
6 ax.set_xscale('log')
7 ax.set_yscale('symlog')
8
```



```

9 line1, = ax.plot(x, expectedValue * x, dashes=[6,2], label='valor
    esperado')
10 line2, = ax.plot(x, y, label='valor gerado')
11
12 ax.legend(handlelength=4)
13 plt.show()

```

Em resumo, a lógica por trás do algoritmo se dá à comparação do valor esperado calculado de uma série de elementos, possuindo seus respectivos pesos probabilísticos, com o valor médio gerado em um milhão de iterações. Quando executado dentro da ferramenta Google Colab para visualização gráfica, obtemos resultados diferentes em cada teste:

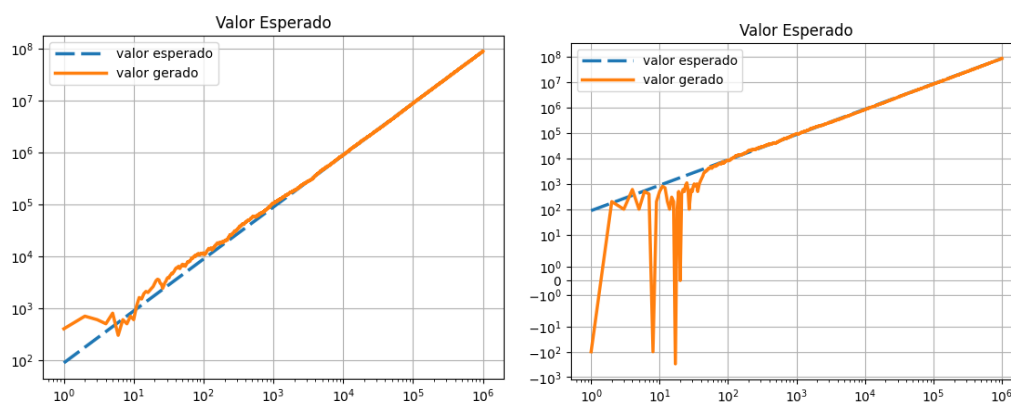


Figura 1. Resultados gráficos do Valor Esperado (Exemplos Iniciais)

Observando atentamente, é possível concluir que, por mais que o valor esperado tenha uma constância, a média dos elementos aleatórios e sua comparação com o valor esperado não são determinísticos, sofrendo de violentas oscilações na linha do valor gerado. Isso se dá ao fato de que com poucos dados gerados, cada sorteio tem um impacto extremo no valor final da média.

Porém, quanto mais iterações são feitas, mais os gráficos tendem a se assemelhar, passando a se alinhar um por cima do outro. Isso se dá em razão da média ter uma tendência a se estabilizar quando há muitos valores, pois cada novo valor aleatório gerado não será capaz de influenciar significativamente no resultado total da média. Sua normalização tende a ser mais perceptível por volta da abscissa 10^2 e da ordenada 10^4 . Após esses pontos, as linhas passam a ficar quase totalmente sobrepostas, possuindo uma diferença cada vez mais insignificante e imperceptível, sendo isso uma consequência da Lei dos Grandes Números.

A Lei dos Grandes Números é um teorema dentro da probabilidade, consistindo na afirmação de que o resultado da realização da mesma experiência repetidas vezes possui a tendência de aproximar o valor consequente da média ao valor esperado. Em outras palavras, quanto mais tentativas são realizadas, mais a probabilidade da média aritmética dos resultados observados irá se aproximar da probabilidade real.

Sendo assim, caso a comparação fosse feita com um número menor de iterações, dependendo da quantidade (por exemplo, 10 iterações), o valor comparado seria extremamente diferente na extrema maioria das vezes.

9. Simulação Computacional - Variância

Dentro da demonstração de variância, continuaremos com o mesmo script usado no valor esperado, demonstrando como a variabilidade se comporta quando se há um grande número de valores sendo calculados, sendo visualizado por gráfico para melhor compreensão visual dos testes.

Com este fim, adicionaremos ao código uma cláusula que irá calcular a variância dos valores e a ilustrar graficamente, primeiramente começando com a implementação dos cálculos da variância, começando primeiro com o cálculo da variância teórica, ou, o valor calculado antes de qualquer simulação:

```
1 # IMPLEMENTANDO COMPARAÇÃO DE VARIÂNCIA
2 # Cálculo da variância teórica
3 theoreticalVariance = 0
4 for i in range(len(values)):
5     # Para cada valor, calcula o desvio ao quadrado em relação à média,
6     # ponderado pela probabilidade
7     theoreticalVariance += ((weights[i] / weightTotal) * ((values[i] -
8     expectedValue) ** 2))
```

Após isso, foi implementado um script para a variância empírica, calculado a partir dos dados reais gerados pela simulação. A variância empírica é uma estimativa da variância teórica baseada em observações:

```
1 # Cálculo da variância empírica ao longo das iterações
2 valueArray = np.array(valueArrayRandom) # Converte a lista para array
3 nArray = np.arange(1, elements + 1) # Array com o número de iterações
4 cumsumValues = np.cumsum(valueArray) # Soma acumulada
5 cumsumSqValues = np.cumsum(valueArray ** 2) # Soma acumulada dos
6     quadrados
7 runningMean = cumsumValues / nArray # Média acumulada a cada iteração
8 runningMeanSq = cumsumSqValues / nArray # Média dos quadrados acumulada
9
10 runningVariance = runningMeanSq - (runningMean ** 2) # Variância empí
11     rica
12 varianceOfMean = runningVariance / nArray # Variância DA MÉDIA
13 theoreticalVarianceOfMean = theoreticalVariance / nArray # Variância te
14     órica da média
```

Assim, com os resultados sendo calculados, chegou a hora de implementar uma ilustração gráfica para os resultados:

```
1 # Implementação do gráfico da variância
2 plt.rc('lines', linewidth=2.5)
3 fig, ax = plt.subplots(figsize=(10, 6))
4
5 ax.set_title('Variância da Média Amostral (Lei dos Grandes Números)')
6 ax.set_xlabel('Número de Iterações (n)')
7 ax.set_ylabel('Variância da Média ( $\sigma^2/n$ )')
8 ax.grid(True)
9 ax.set_xscale('log') # Escala log no eixo X para visualizar todas as
10     iterações
11 ax.set_yscale('log') # Escala log no eixo Y pois a variância decai
12     exponencialmente
```

```

11
12 # Começa no índice 10 para evitar instabilidade numérica nas primeiras
    iterações
13 start = 10
14
15 # Linha tracejada: variância teórica da média, o comportamento ideal
    esperado
16 ax.plot(nArray[start:], theoreticalVarianceOfMean[start:], dashes
    =[6,2], label='sigma^2/n (teórico)')
17
18 # Linha contínua: variância empírica da média calculada a cada iteração
19 ax.plot(nArray[start:], varianceOfMean[start:], label='variância empí
    rica da média', alpha=0.7)
20
21 ax.legend(handlelength=4)
22 plt.tight_layout()
23 plt.show()

```

Como resultados, foram obtidas as seguintes variâncias para cada caso gerado, ilustrando a comparação entre o valor esperado com o valor gerado em cima, seguidos com o gráfico da variância dos mesmos valores logo abaixo:

Caso 1

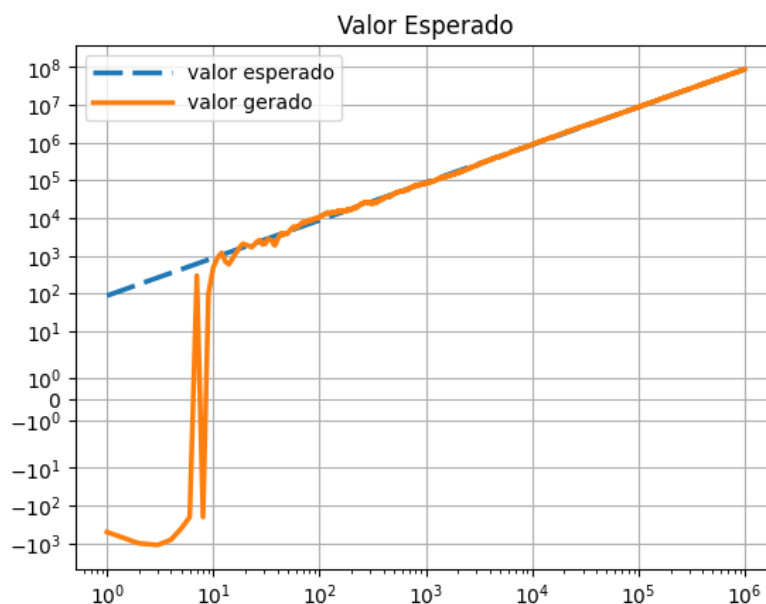


Figura 2. Caso 1: Comparação do Valor Esperado

Caso 2

Caso 3

Com isso, é possível observar que, tal como na variância, há uma primeira oscilação no cálculo da variância baseada nos valores aleatórios. Por mais que a oscilação não aparente ser tão brusca na ilustração, ela é que tende a oscilação gerada no gráfico do valor esperado e da média geral. Adiante, as linhas tendem a também se normalizar, chegando a estarem

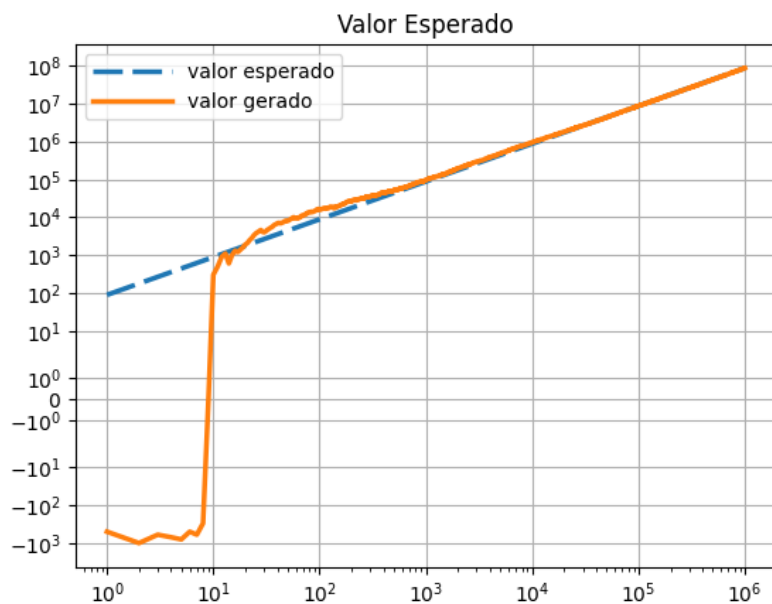


Figura 3. Caso 1: Variância da Média Amostral

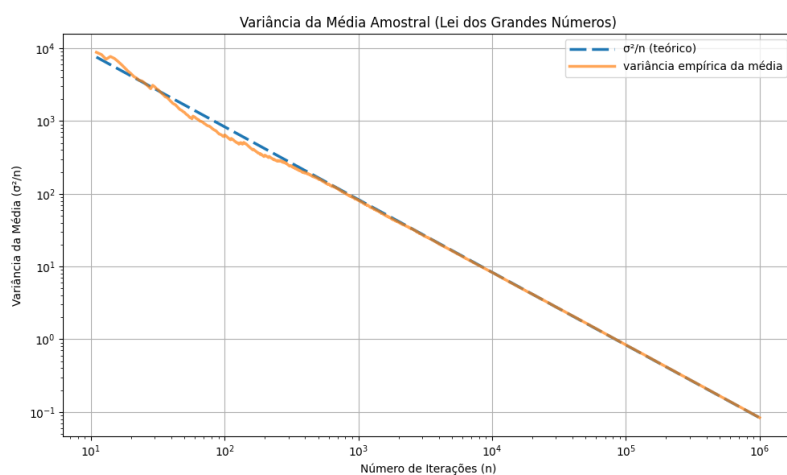


Figura 4. Caso 2: Comparação do Valor Esperado

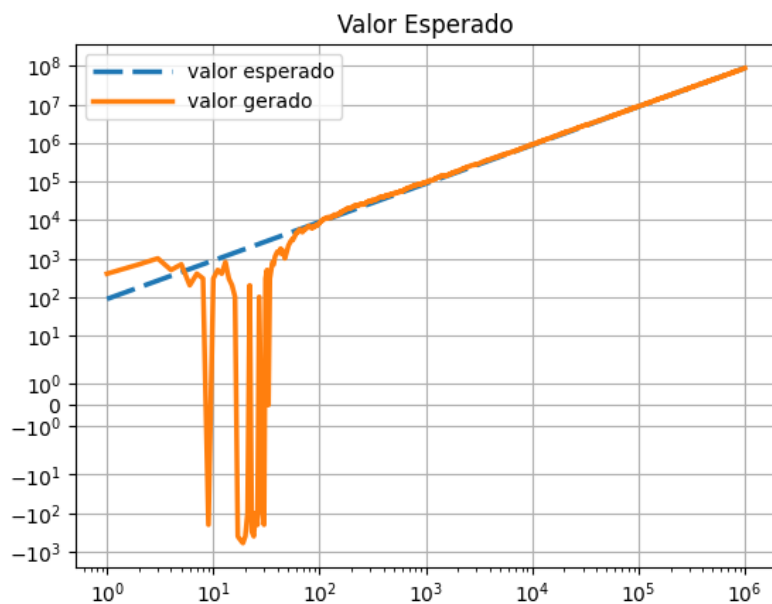


Figura 5. Caso 2: Variância da Média Amostral

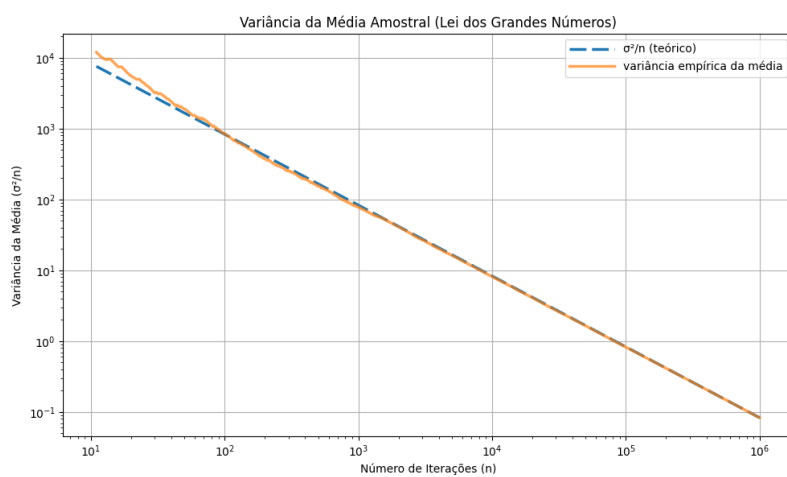


Figura 6. Caso 3: Comparação do Valor Esperado

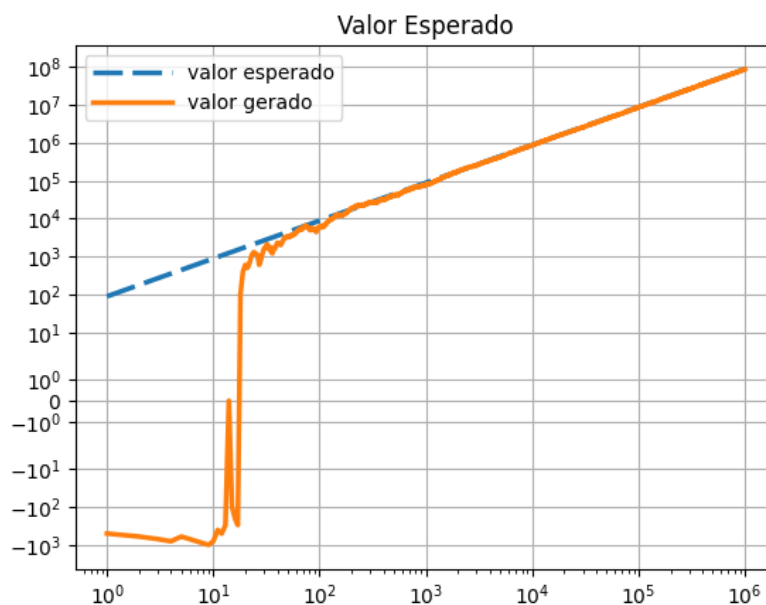


Figura 7. Caso 3: Variância da Média Amostral

ambas uma sobre a outra, porém, além dessa semi igualdade, o gráfico, com o passar da série de simulações, segue diminuindo o seu valor, chegando cada vez mais próximo de zero, porém, nunca em zero. Isso se dá ao fato de que, com a Lei dos Grandes Números, a variância tende a praticamente zerar em relação ao valor esperado, pois, como maiores testes levam a resultados cada vez mais perto da média, o alcance de variabilidade também diminui, levando-o a seguir uma constância.

10. Conclusão

A execução deste trabalho permitiu comprovar de forma objetiva e prática a profunda interdependência teórica existente entre os conceitos matemáticos abordados. Através das simulações computacionais desenvolvidas e analisadas, evidenciou-se que essas ferramentas formam um ciclo contínuo: a Teoria da Probabilidade baseia o cenário inicial mapeando as chances, o Teorema de Bayes atualiza a nossa visão assim que novas evidências surgem, o Valor Esperado direciona a tomada de decisão ao calcular tendências a longo prazo, e a Variância alerta medindo a instabilidade e o risco real da operação. Fica claro, portanto, que utilizar essas métricas de maneira isolada compromete a fidelidade da análise estatística.

Diante dos resultados obtidos, conclui-se sobre a indiscutível importância prática de se dominar esses quatro fundamentos estatísticos em conjunto. A compreensão integrada dessas métricas é absolutamente essencial para a tomada de decisões reais e seguras em qualquer projeto computacional ou empresarial que envolva cenários de incerteza. Ao abandonar o empirismo e aplicar essa engrenagem metodológica, desenvolvedores e engenheiros tornam-se capazes de mensurar riscos de forma assertiva, evitando armadilhas matemáticas e garantindo a construção de sistemas lógicos, eficientes e robustos diante do imprevisível.

Referências

- CALCULATOR SOUP. Variance calculator. Acesso entre: 20 jun. 2026 e 5 jul. 2026.
- CUEMATH. Sample variance formula - cuemath. Acesso entre: 20 jun. 2026 e 5 jul. 2026.
- da Silva, M. E. Uma nota sobre esperança condicional e expectativas racionais. Acesso entre: 20 jun. 2026 e 5 jul. 2026.
- de Farias, A. M. L. and da Costa Laurencel, L. (2006). *Probabilidade*. Universidade Federal Fluminense, Niterói. Acesso entre: 20 jun. 2026 e 5 jul. 2026.
- Ferrari. Esperança condicional - ferrari. Acesso entre: 20 jun. 2026 e 5 jul. 2026.
- KANARIES. Variance calculator. Acesso entre: 20 jun. 2026 e 5 jul. 2026.
- Salonen, J. (2013). Deriving welford's method for computing variance. Acesso entre: 20 jun. 2026 e 5 jul. 2026.
- Smith, A. and Jones, B. (1999). On the complexity of computing. In Smith-Jones, A. B., editor, *Advances in Computer Science*, pages 555–566. Publishing Press.
- UNIVERSIDADE FEDERAL DE SERGIPE. Aula 07. Acesso entre: 20 jun. 2026 e 5 jul. 2026.
- UNIVERSIDADE FEDERAL DE SERGIPE. Probabilidade e estatística: Aula 11. Acesso entre: 20 jun. 2026 e 5 jul. 2026.
- Valdés, J. T. Probabilidade i - probabilidade e variáveis aleatórias unidimensionais. Acesso entre: 20 jun. 2026 e 5 jul. 2026.
- WIKIPEDIA CONTRIBUTORS. Algorithms for calculating variance. In: Wikipedia, The Free Encyclopedia. Acesso entre: 20 jun. 2026 e 5 jul. 2026.
- WIKIPEDIA CONTRIBUTORS. Lei dos grandes números - wikipédia. Acesso: 29 abr. 2026.